Utilizando sockets com Java

Sistemas Distribuídos

Eduardo Furlan Miranda

2025-10-07

Adaptado de: PEREIRA, C. S. Sistemas Distribuídos.
Londrina: EDE SA, 2019. ISBN 978-85-522-1443-4.

SO e Sockets

- O mecanismo de Sockets é uma parte fundamental e nativa do kernel (o núcleo) do Windows, Linux, macOS, BSD e outros
- É um serviço central que o sistema oferece para qualquer aplicação que precise de rede
- Um programa "chama o sistema operacional" usando uma API que expõe um conjunto de funções (que são, em baixo nível, "chamadas de sistema" ou system calls) que um programador pode usar

Linux

- A API: chama-se Sockets POSIX

- Como funciona
  - É o padrão da indústria
  - Funções como socket(), bind(), listen(), connect(), send() e recv() são parte da biblioteca C padrão (libc)
  - Quando um programa em C chama essas funções, ele está, na prática, pedindo ao kernel do Linux para realizar essas operações de rede

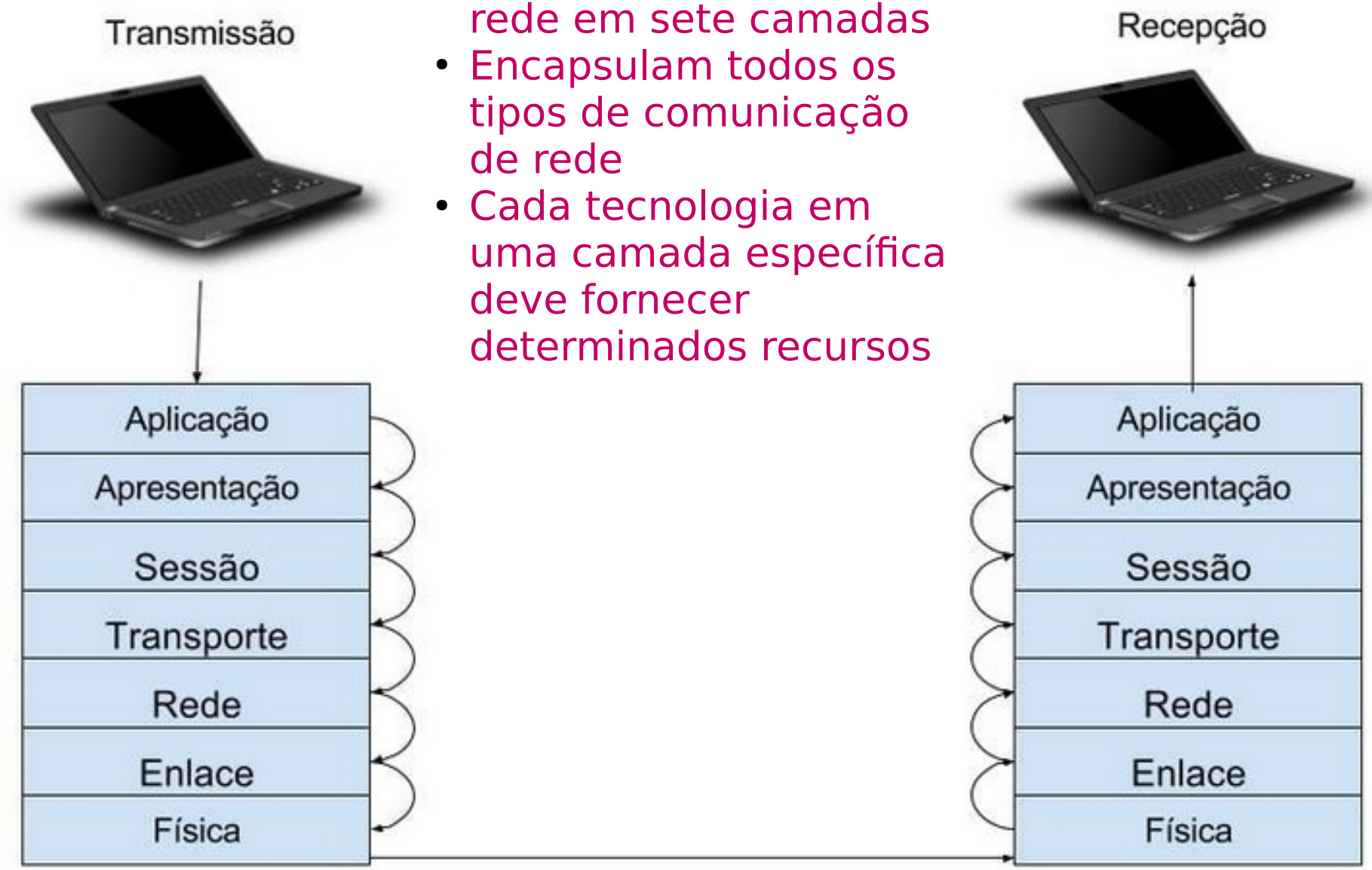
Windows

- A API: Chama-se Winsock (Windows Sockets API)

- Como funciona
  - O Winsock foi projetado pela Microsoft para ser muito parecido com o padrão de Sockets POSIX, para que os programas pudessem ser facilmente portados do UNIX para o Windows
  - As funções têm os mesmos nomes (socket(), bind(), etc.)

7 camadas ISO/OSI

- Estrutura conceitual que divide as funções de comunicação de rede em sete camadas
- Encapsulam todos os tipos de comunicação de rede
- Cada tecnologia em uma camada específica deve fornecer determinados recursos



OSI

- A comunicação dos dados de uma máquina para outra ocorre a partir da quarta camada (a chamada camada de transporte)
- Destacam-se dois protocolos muito utilizados para realizar tal comunicação: o protocolo TCP e o protocolo UDP
- TCP: caso haja alguma perda de informação durante a transmissão, a informação é retransmitida
- UDP: mais simples, rápido, porém não retransmite

Sockets

- Mecanismo de comunicação de rede
- Enviar e receber dados de forma bidirecional entre diferentes processos, na mesma máquina ou em diferentes
- Usa o protocolo TCP para realizar a comunicação entre aplicações que estejam sendo executadas em um sistema operacional
- Usa IP:Porta (ex.: 192.168.1.10:8080)
- Modelo cliente-servidor

Primitivas de sockets TCP

Significado	Significado
Socket	Cria um novo terminal de comunicação.
Bind	Atrela um endereço IP local a um *socket*.
Listen	Aviso de que o socket está aceitando conexões.
Accept	Aguarda o recebimento de uma solicitação de conexão.
Connect	Ativamente tenta estabelecer conexão com um *socket*.
Send	Envia dados através de uma conexão previamente estabelecida.
Receive	Recebe dados através de uma conexão previamente estabelecida.
Close	Libera a conexão.

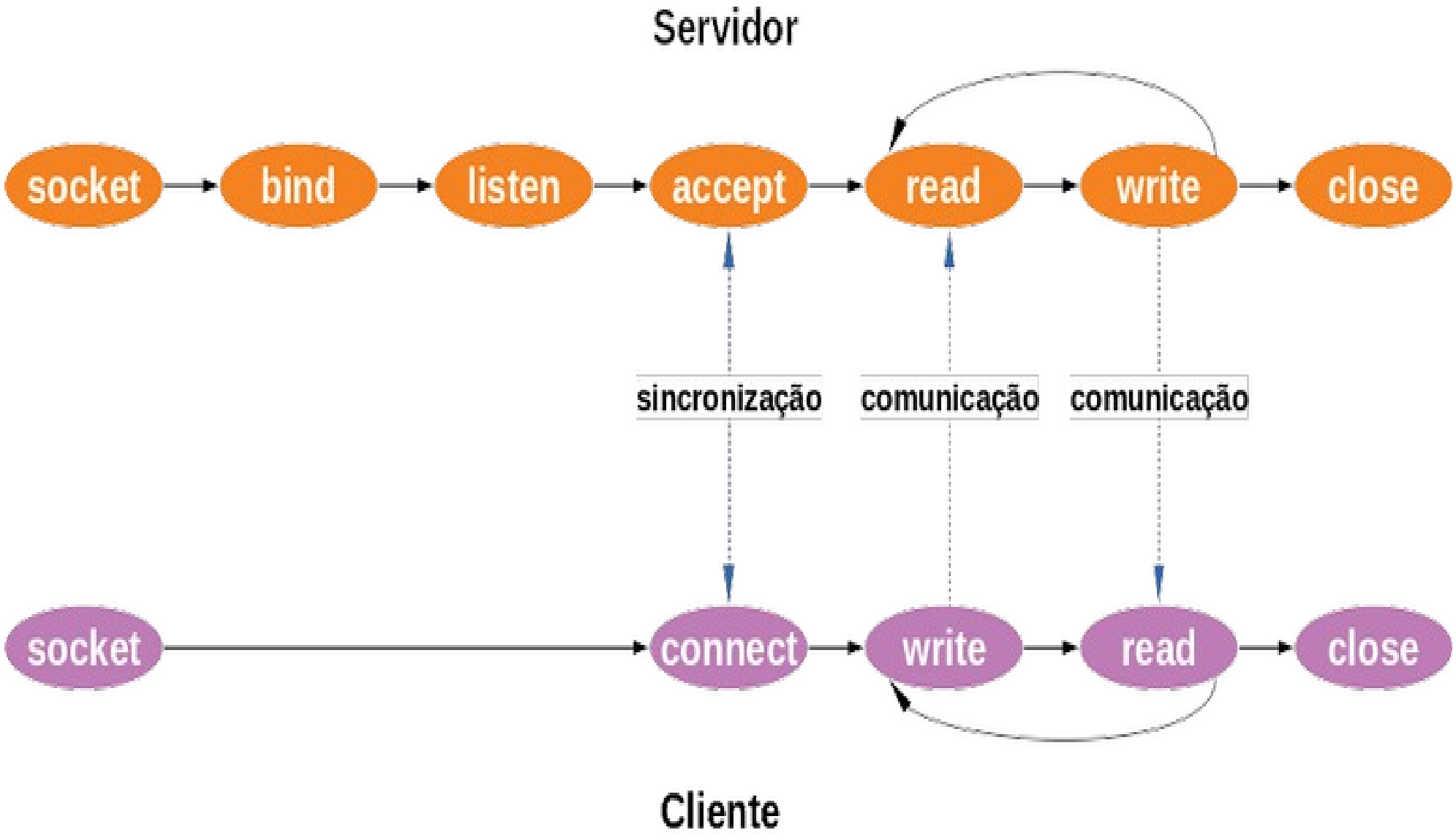
No servidor

- A primitiva “socket” cria um novo terminal de comunicação
- Internamente o sistema operacional aloca os recursos necessários
- “bind” associa um socket específico ao servidor, de forma que o sistema operacional saiba
- “listen” faz com que o sistema operacional reserve recursos
- “accept” fica aguardando o estabelecimento de uma conexão
  - Quando esta ocorre, o sistema operacional retorna um socket com as mesmas características daquele criado pelo servidor

Nos clientes

- Cria-se um socket, mas não há necessidade de especificar uma porta de comunicação
- “connect” recebe a informação de qual protocolo de transporte foi adotado pelo servidor e fica aguardando o estabelecimento de uma conexão
- “send” e “receive” troca informações ou mensagens
- “close” explicitamente encerra e libera os recursos

Processo de comunicação via sockets TCP



Servidor

```
1 import java.io.DataInputStream;
2 import java.io.DataOutputStream;
3 import java.io.IOException;
4 import java.net.ServerSocket;
5 import java.net.Socket;
6
7 public class Servidor {
8
9         public static void main(String[] args) {
10                Socket soc = null;
11                ServerSocket socServidor = null;
12                try {
13                        socServidor = new ServerSocket(5001);
14                        soc = socServidor.accept();
15                        DataInputStream recebido = new DataInputStream(soc.getInputStream());
16                        DataOutputStream enviado = new DataOutputStream(soc.getOutputStream());
17                        System.out.println("(cliente): " + recebido.readUTF());
18                        enviado.writeUTF("O servidor recebeu sua mensagem.");
19                } catch (IOException ex) {
20                        System.err.println("Falha na conexão");
21                } finally {
22                        try {
23                                soc.close();
24                                socServidor.close();
25                        } catch (IOException e) {
26                                System.err.println("Falha ao encerrar a conexão");
27                        }
28                }
29        }
30 }
```

Cliente

```
1 import java.io.DataInputStream;
2 import java.io.DataOutputStream;
3 import java.io.IOException;
4 import java.net.Socket;
5
6 public class Cliente {
7
8         public static void main(String[] args) {
9                 Socket soc = null;
10                try {
11                        soc = new Socket("127.0.0.1", 5001);
12                        DataInputStream recebido = new DataInputStream(soc.getInputStream());
13                        DataOutputStream enviado = new DataOutputStream(soc.getOutputStream());
14                        enviado.writeUTF("Aqui é um cliente falando...");
15                        System.out.println("(servidor): " + recebido.readUTF());
16                } catch (IOException ex) {
17                        System.err.println("Falha na inicializar o servidor");
18                } finally {
19                        try {
20                                soc.close();
21                        } catch (IOException e) {
22                                System.err.println("Falha ao encerrar a conexão");
23                        }
24                }
25        }
26 }
```